# Bring Your Own Git (BYOG) — Edge Delivery Services with AEM authoring

**One codebase in an Adobe-hosted Cloud Manager repository, many Edge Delivery sites (dev / QA / stage / prod), each pointing at its own AEM author environment.**

| | |
|---|---|
| Organization (aem.live org) | `gehealthcare-eds` |
| Canonical site / repo | `geh-brand` |
| Environment sites (examples) | `geh-brand-qa`, `geh-brand-stage` |
| Cloud Manager program id | `<program-id>` |
| Cloud Manager repository id | `<repository-id>` |
| AEM author (canonical) | `https://author-p<program-id>-e<env-id>.adobeaemcloud.com` |
| AEM author (QA example) | `https://author-p<program-id>-e<qa-env-id>.adobeaemcloud.com` |

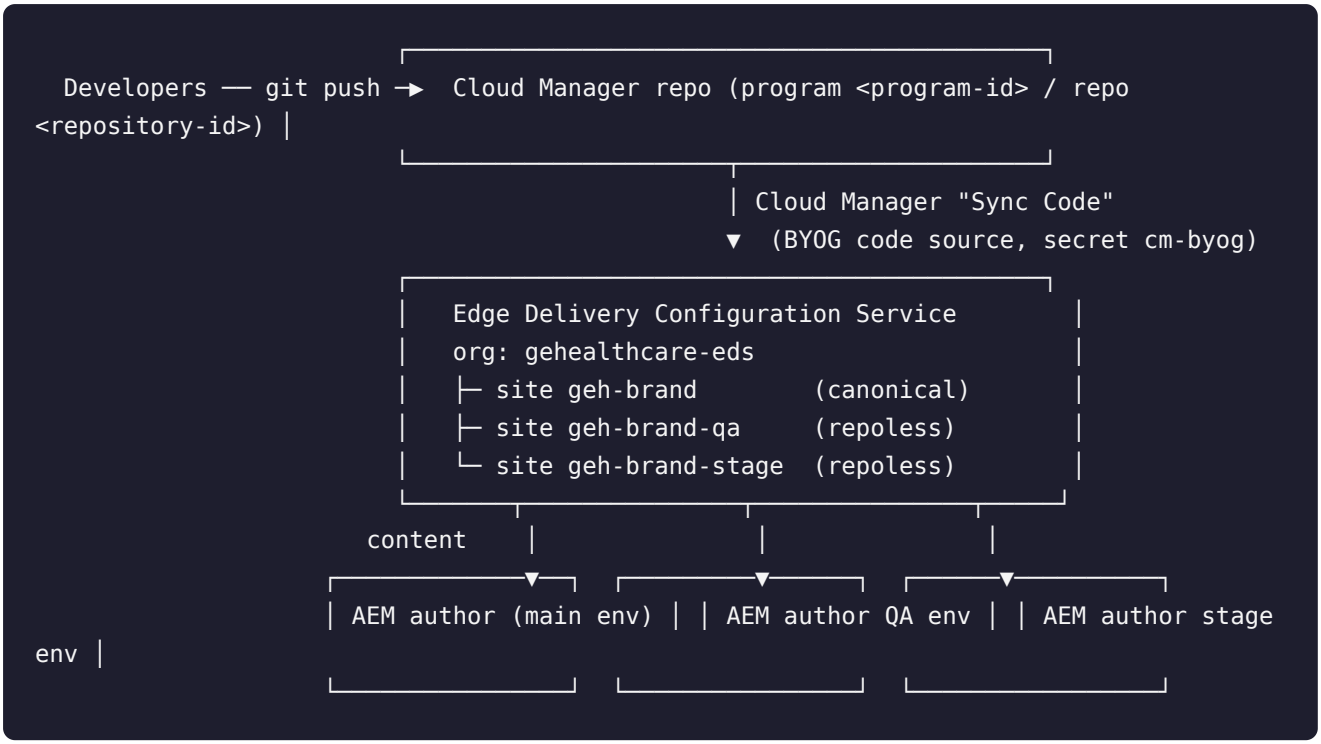
Official references used throughout (all screenshots in this guide come from these pages):

- Developer tutorial (AEM authoring / Universal Editor): https://www.aem.live/developer/ue-tutorial
- Bring Your Own Git: https://www.aem.live/developer/byo-git
- Repoless — one codebase, many sites: https://www.aem.live/docs/repoless
- Configuration Service setup (Admin API): https://www.aem.live/docs/config-service-setup
- Admin API keys: https://www.aem.live/docs/admin-apikeys

> **Security:** never paste an Admin API token or Cloud Manager secret into chat tools, tickets or Git. Keep them in a password manager or an environment variable ( `AEM_TOKEN` ) and rotate them regularly.

---

## How the pieces fit together




- **Code** lives once, in the Cloud Manager repository. Every site points to the same program id + repository id.
- **Content** is configured per site: each site's `content.source.url` points to the `franklin.delivery` endpoint of its own AEM author environment.
- **Site URLs** follow `https://<branch>--<site>--<org>.aem.page` (preview) and `.aem.live` (live), e.g. `https://main--geh-brand-qa--gehealthcare-eds.aem.live/` .

## Prerequisites

- A **github.com organization named** `gehealthcare-eds` that GE controls. Even when the code lives in Cloud Manager, the aem.live org name must exist on github.com so nobody else can claim it.

- A GitHub account with Git basics, Node.js/npm, and the AEM Sidekick extension.
- Access to the AEM as a Cloud Service author environments and to **Cloud Manager** for program `<program-id>` (a role that can manage repositories and Edge Delivery sites, e.g. Deployment Manager / Business Owner).
- An **Admin API token** with admin rights on the org/site (see Part D).

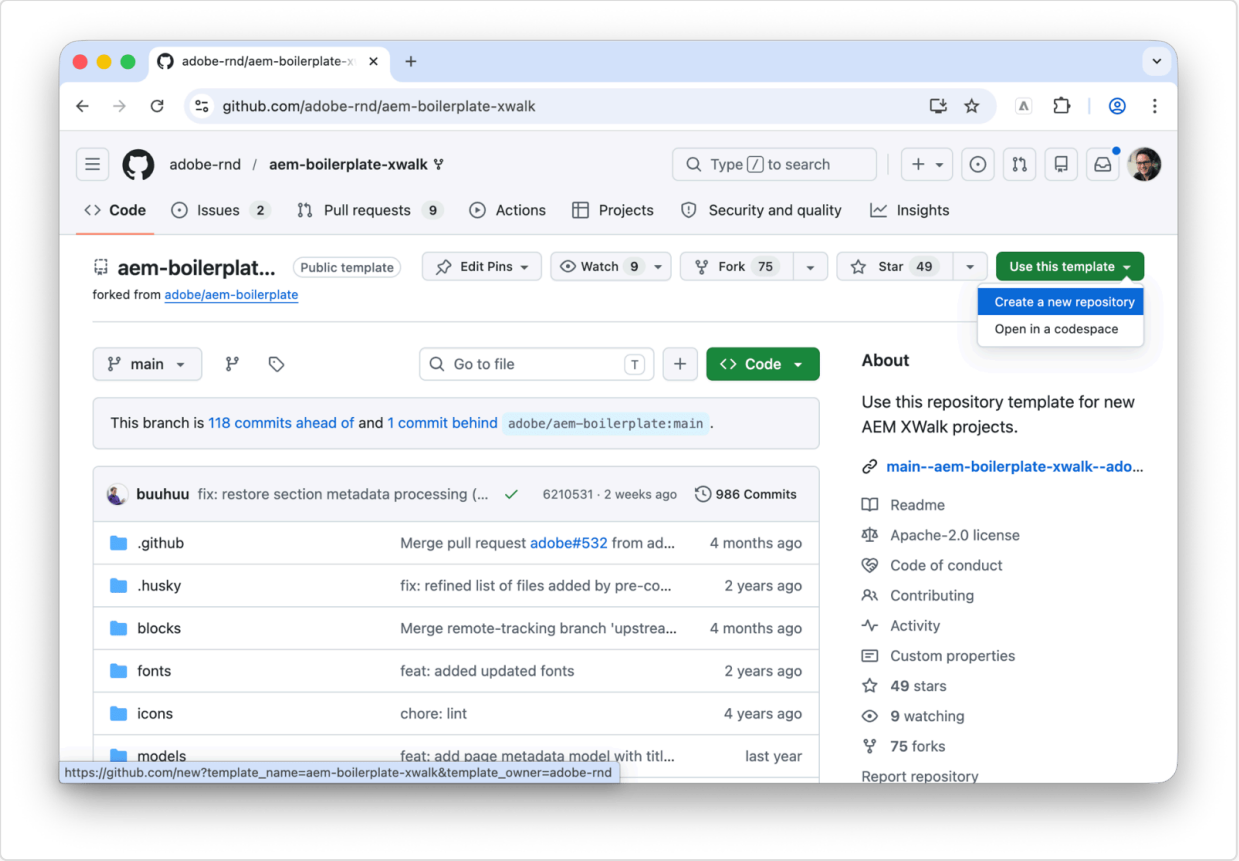
---

# Part A — Create the canonical site from the boilerplate (GitHub)

This bootstraps the org and the first site in the configuration service. It follows the official developer tutorial.

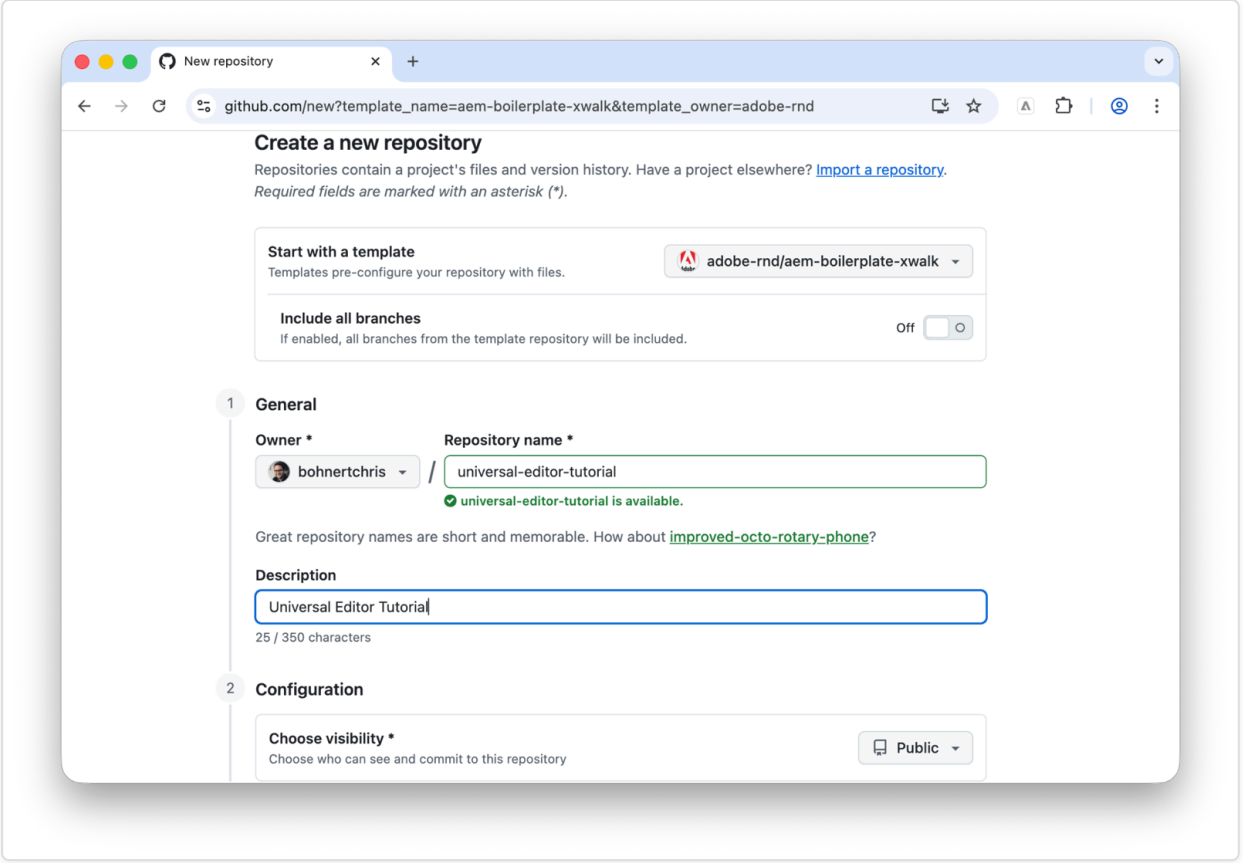
## A1. Create the repository from the xwalk boilerplate

1. Sign in to github.com and open the AEM authoring boilerplate: https://github.com/adobe-rnd/aem-boilerplate-xwalk.
2. Click **Use this template → Create a new repository**.




*Source: aem.live developer tutorial*

3. Choose owner **`gehealthcare-eds`**, name the repository **`geh-brand`**, add a description and click **Create repository**.

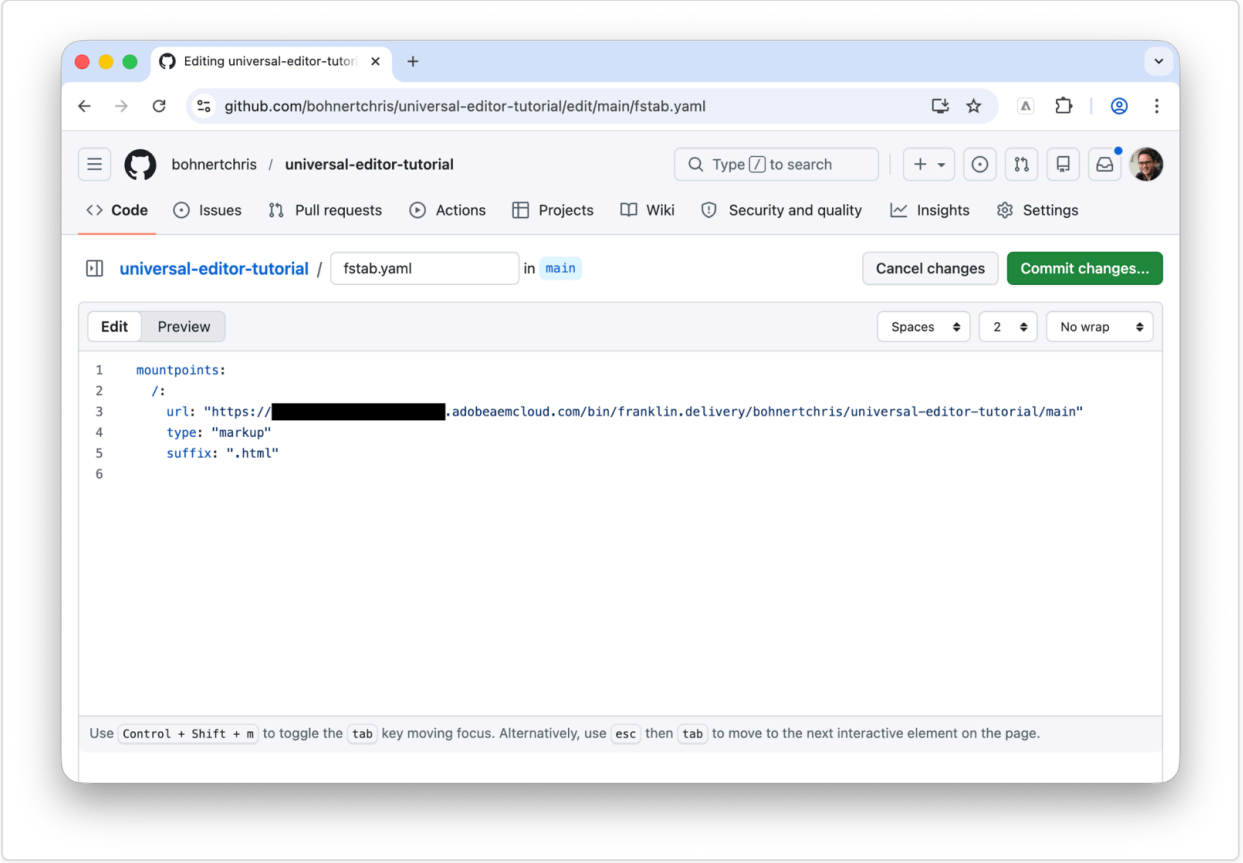



*Source: aem.live developer tutorial*

## A2. Point `fstab.yaml` at the AEM author

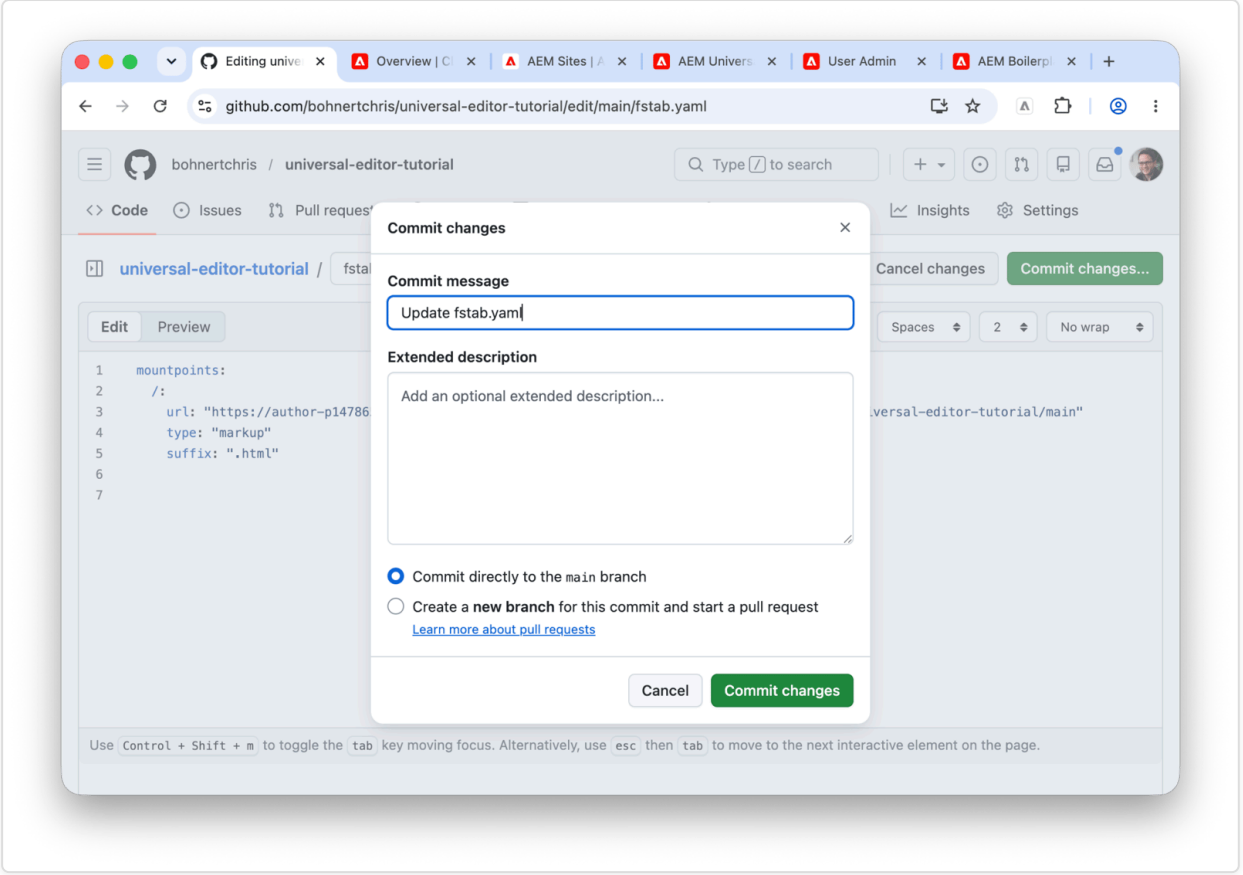
1. In the new repository open `fstab.yaml` and click the pencil (**Edit this file**).
2. Replace the mount point with the AEM author `franklin.delivery` URL (format `https://<aem-author>/bin/franklin.delivery/<owner>/<repository>/main`):

```
mountpoints:
  /:
    url: "https://author-p<program-id>-e<env-id>.adobeaemcloud.com/bin/franklin.delivery/gehealthcare-eds/geh-brand/main"
    type: "markup"
    suffix: ".html"
```




*Source: aem.live developer tutorial*

3. Click **Commit changes…**, add a message and commit **directly to** `main`.

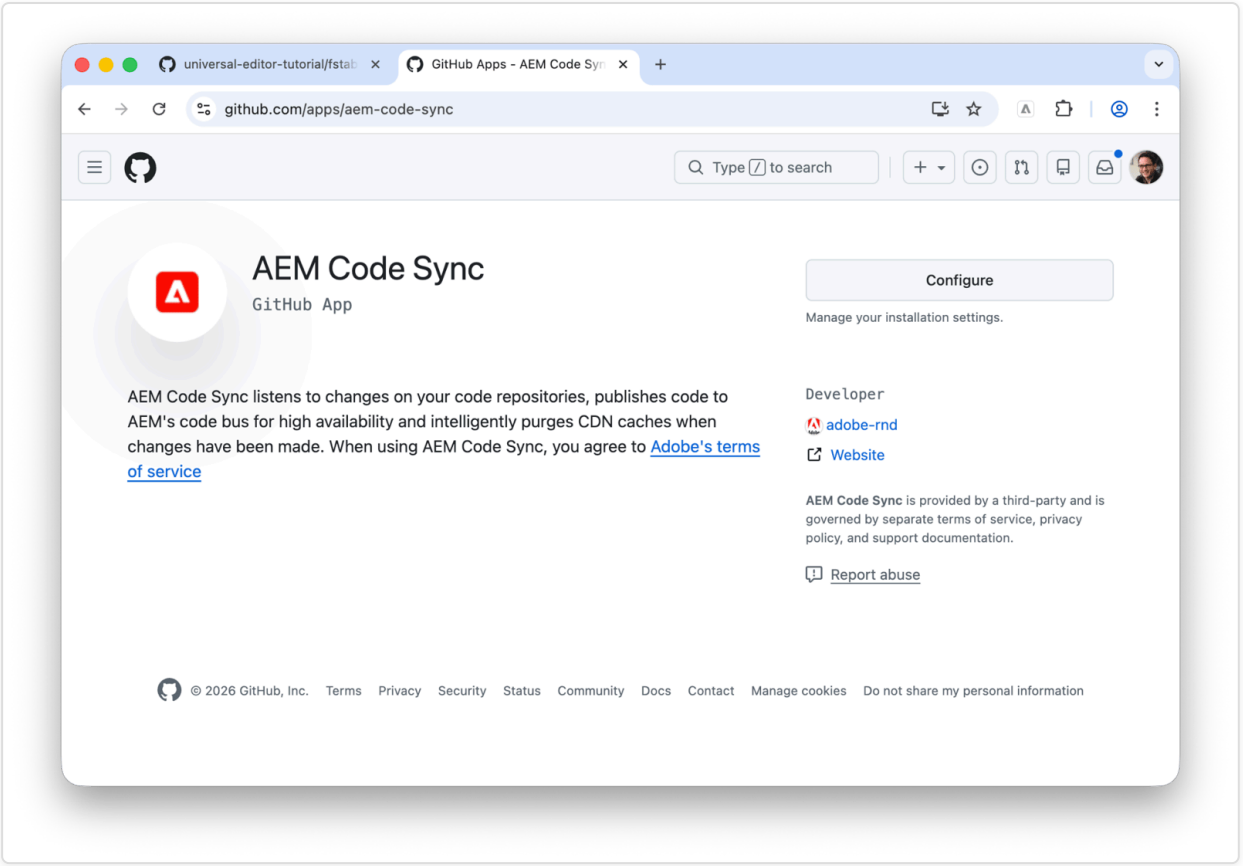



*Source: aem.live developer tutorial*

> Update `fstab.yaml` **before** installing AEM Code Sync. Once Code Sync has written the site into the configuration service, `fstab.yaml` is no longer read; later content-source changes are made in the configuration service (Site Admin tool or Admin API — see Part E).

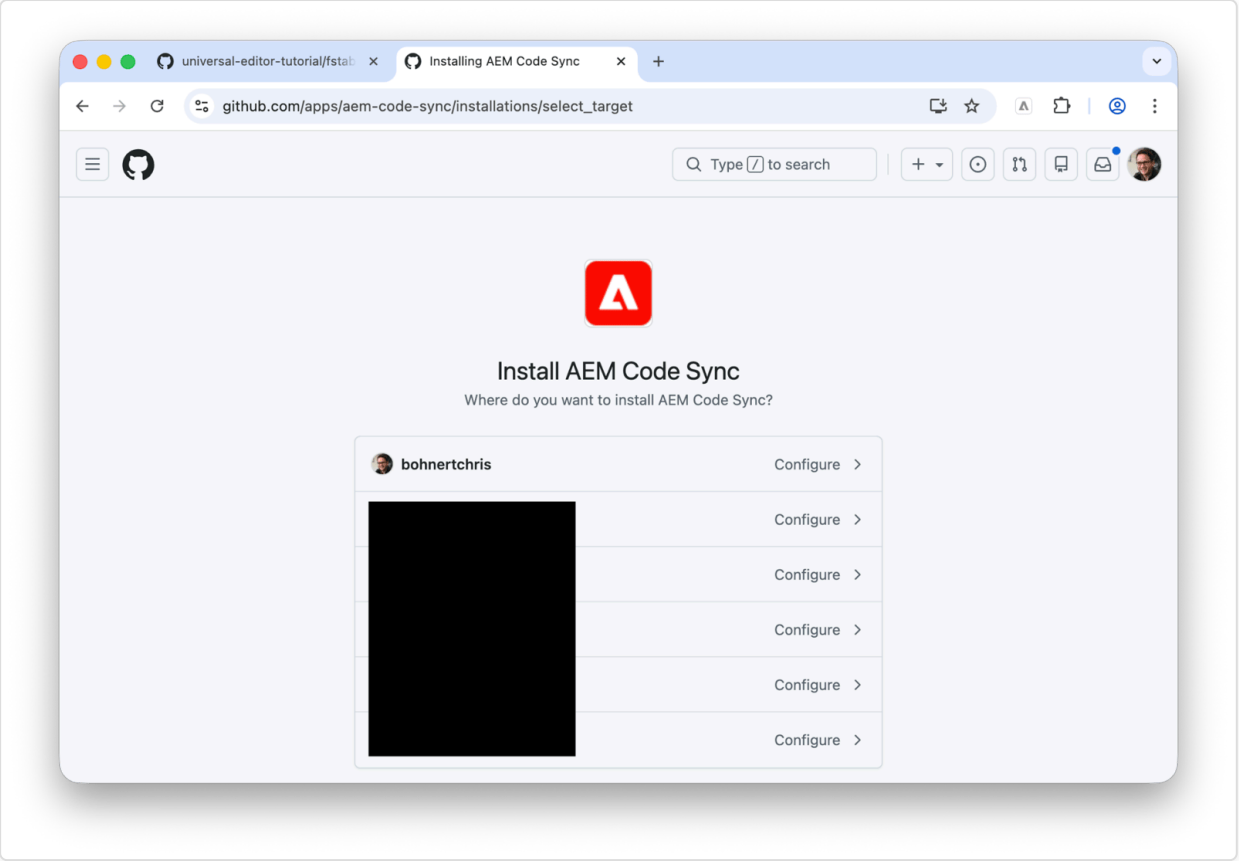
## A3. Install AEM Code Sync

1. In the same browser open https://github.com/apps/aem-code-sync and click **Configure**.



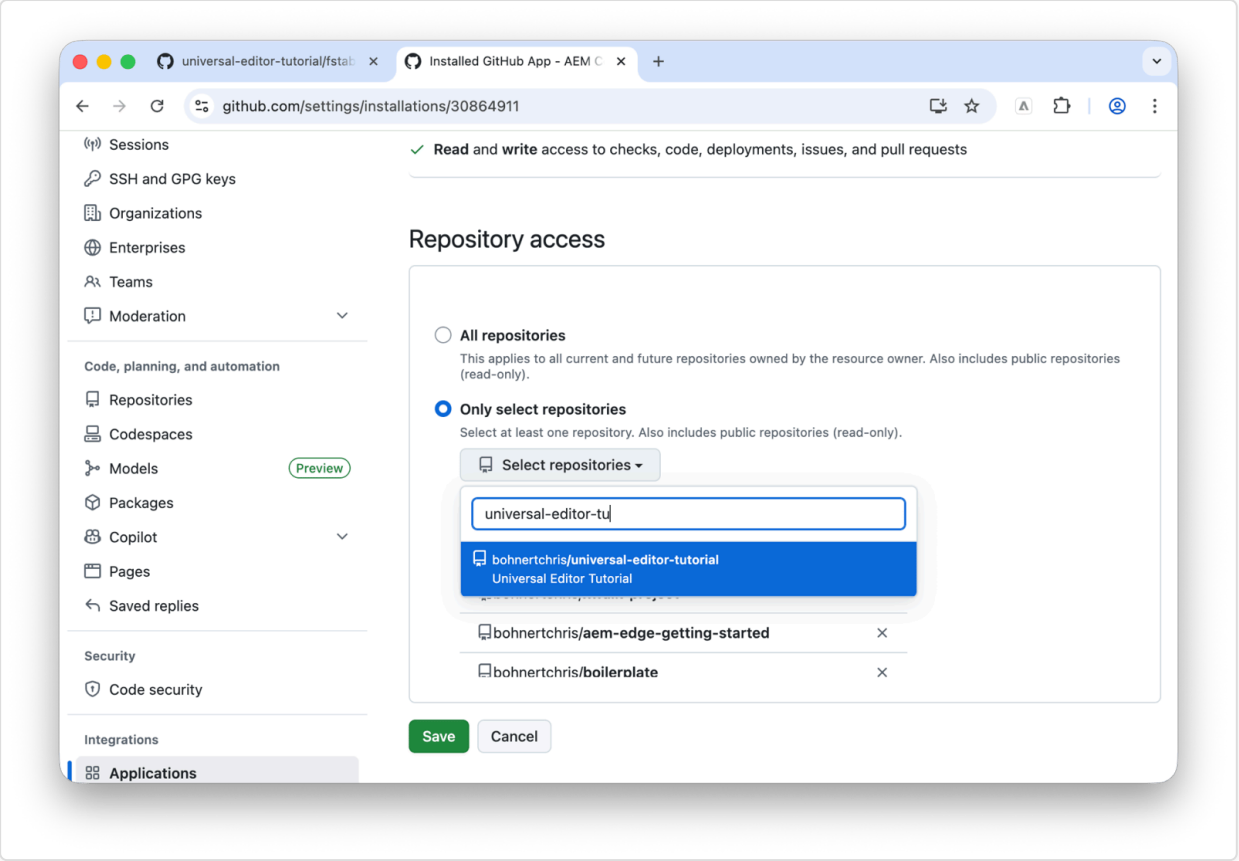

*Source: aem.live developer tutorial*

2. Pick the **`gehealthcare-eds`** organization.




*Source: aem.live developer tutorial*

3. Under **Repository access** choose **Only select repositories**, select `geh-brand` and **Save**.

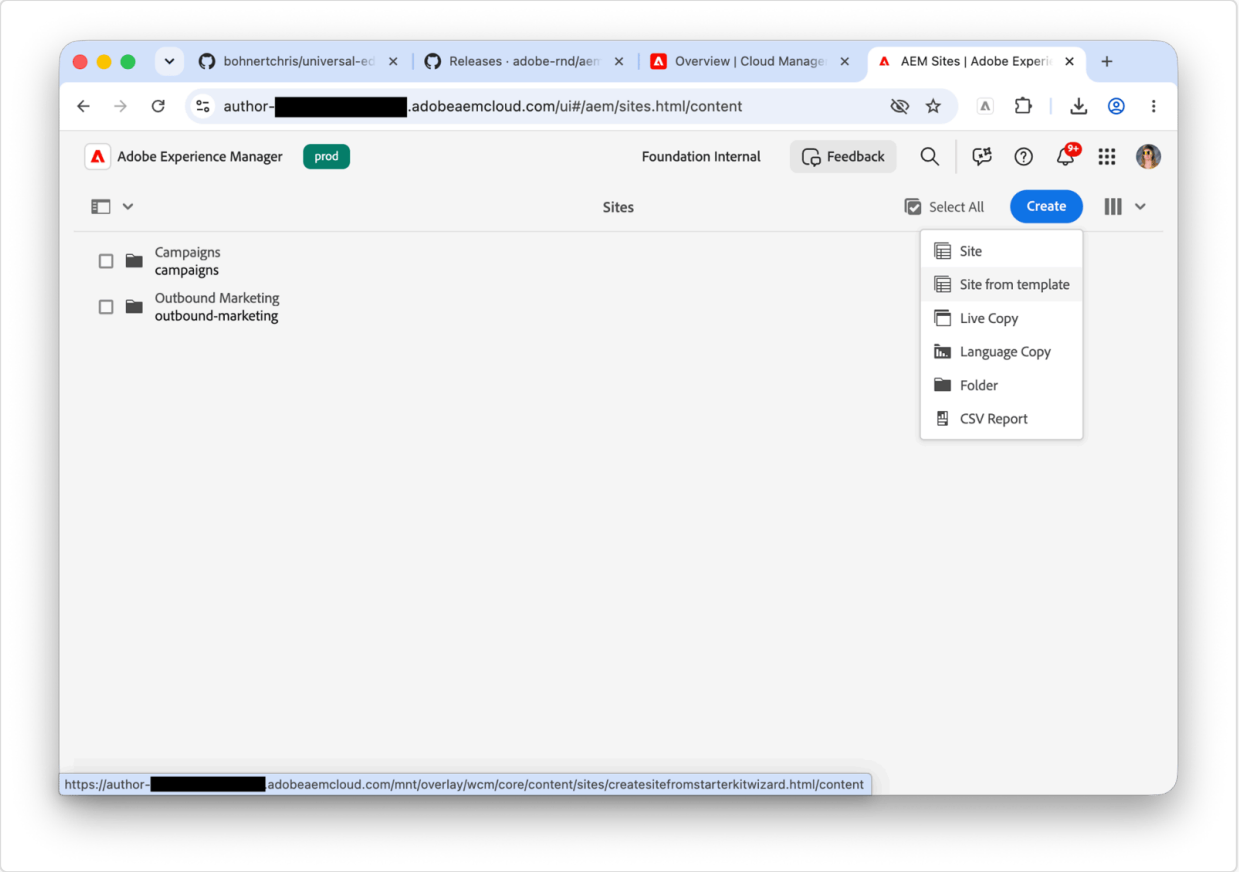



*Source: aem.live developer tutorial*

4. Complete the **AEM Configuration Setup** wizard that opens:
    - **Code** → Next.
    - **Content** → type is pre-set to *Other (bring your own markup)* and the URL is read from `fstab.yaml` → Next.
    - **Users** → add the admin users for this site configuration (your own e-mail first!) → Next.
    - **Finish** → review and **Save**.
5. Verify the code is served: https://main--geh-brand--gehealthcare-eds.aem.page/scripts/aem.js should return JavaScript.

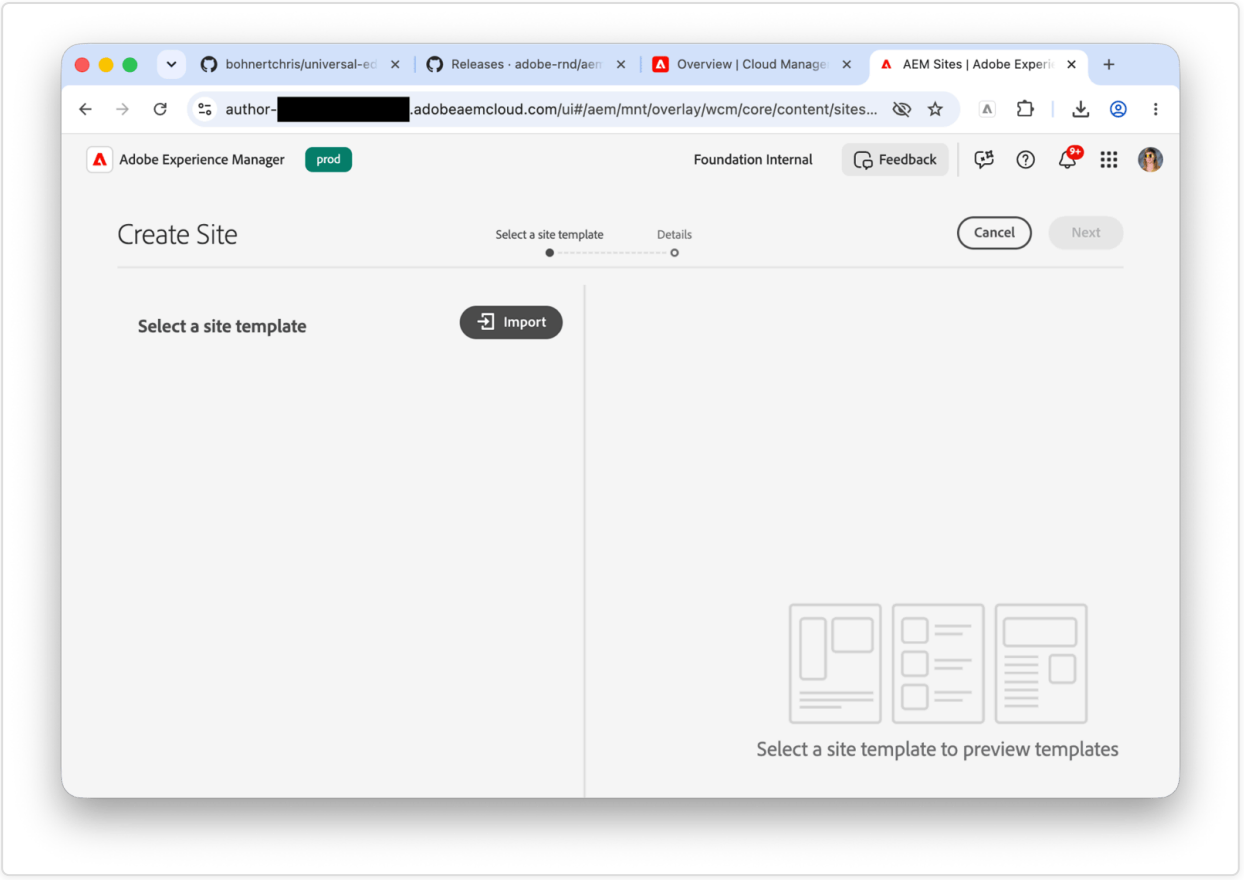
## A4. Create the AEM site from the template

1. Download the latest site template ( `.zip` ) from https://github.com/adobe-rnd/aem-boilerplate-xwalk/releases.
2. In the AEM author **Sites** console click **Create → Site from template**.



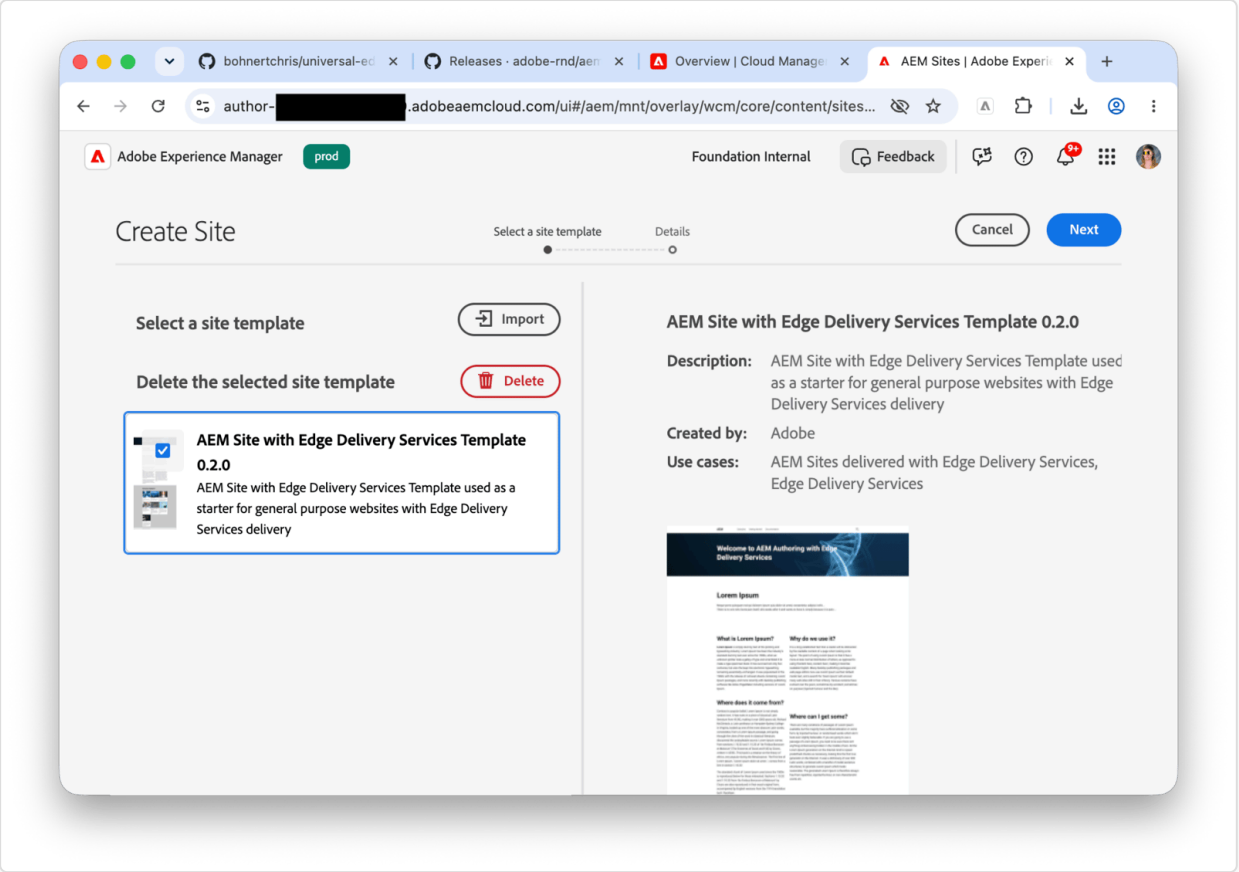

*Source: aem.live developer tutorial*

3. Click **Import** and upload the template (only needed once per environment; it can be reused for more sites).



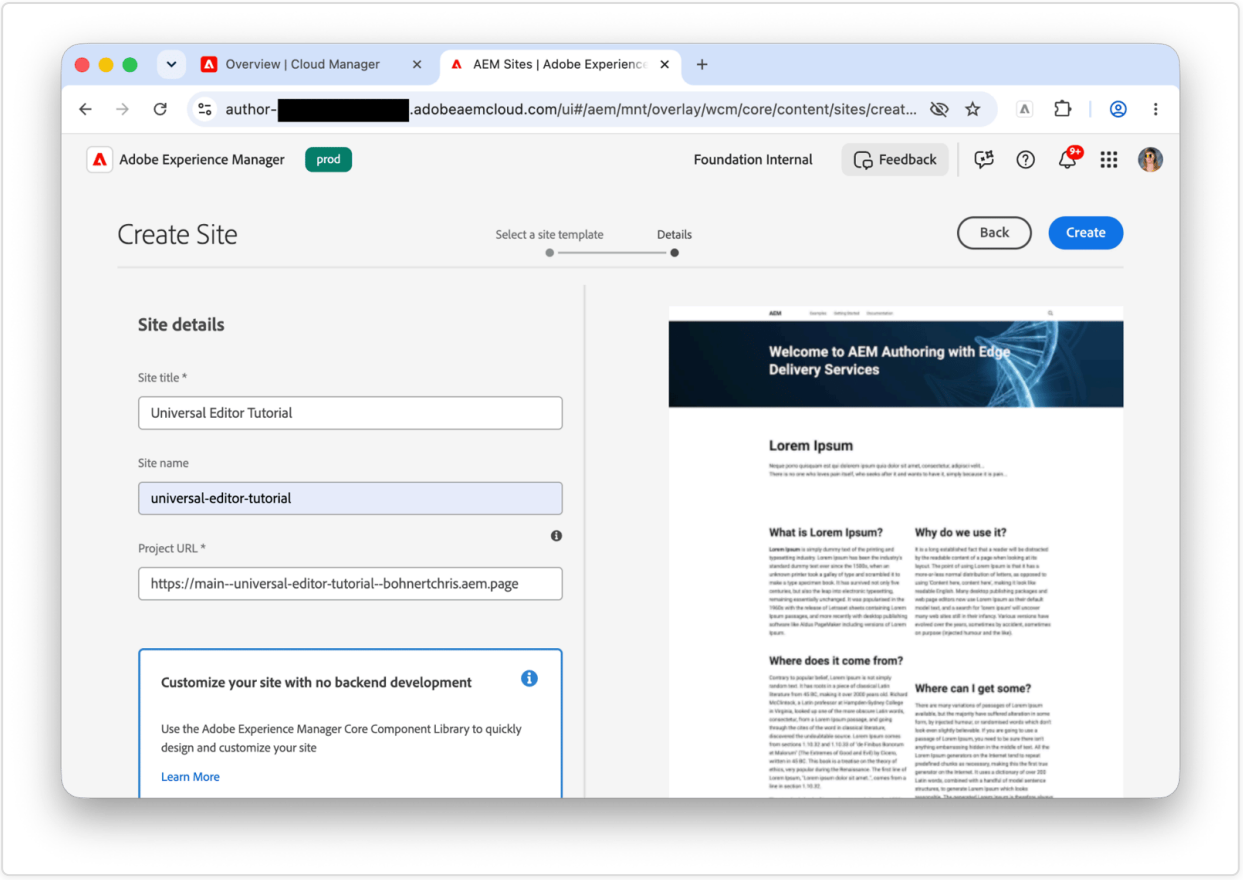

*Source: aem.live developer tutorial*

4. Select the imported template → **Next**.



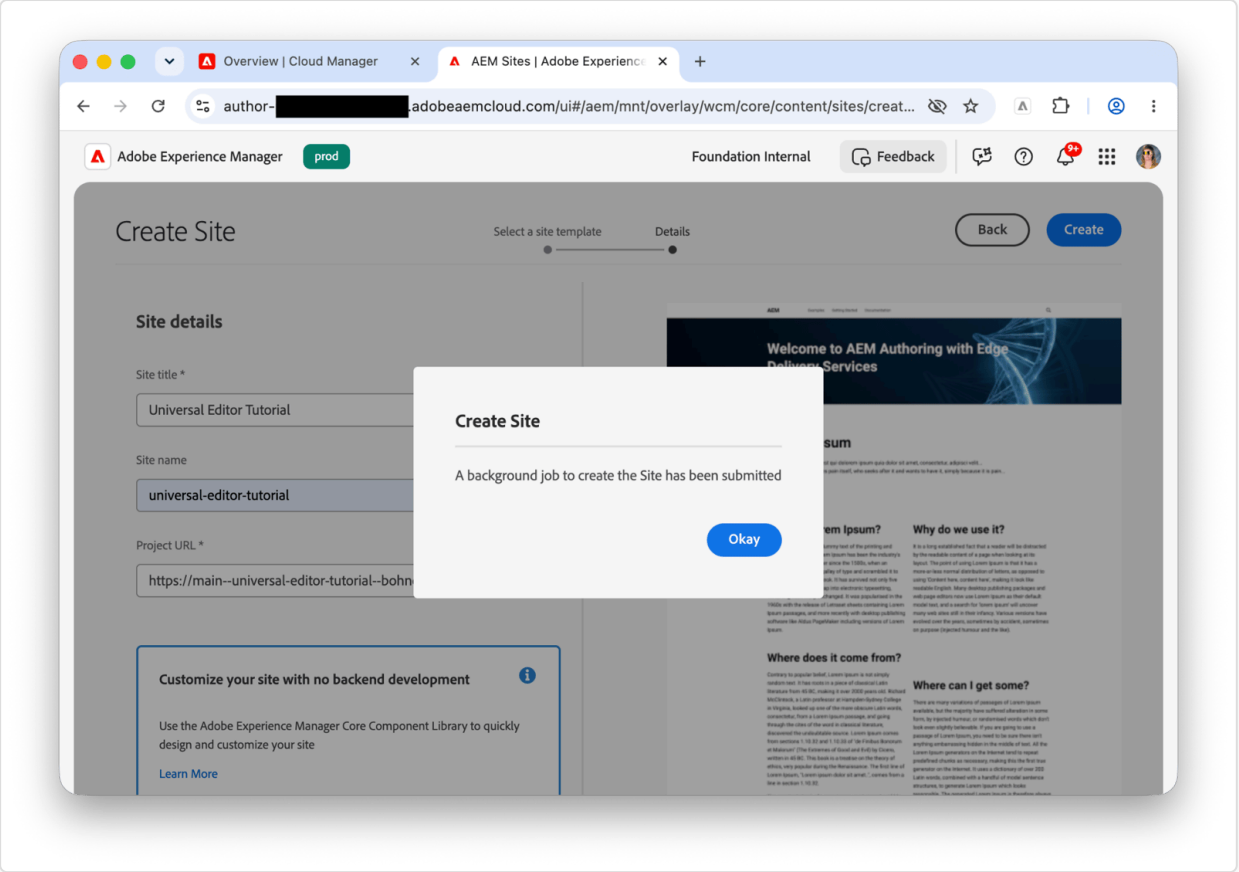

*Source: aem.live developer tutorial*

5. Fill in **Site title**, **Site name** ( `geh-brand` ) and **Project URL** `https://main--geh-brand--gehealthcare-eds.aem.page` , then **Create**.




*Source: aem.live developer tutorial*

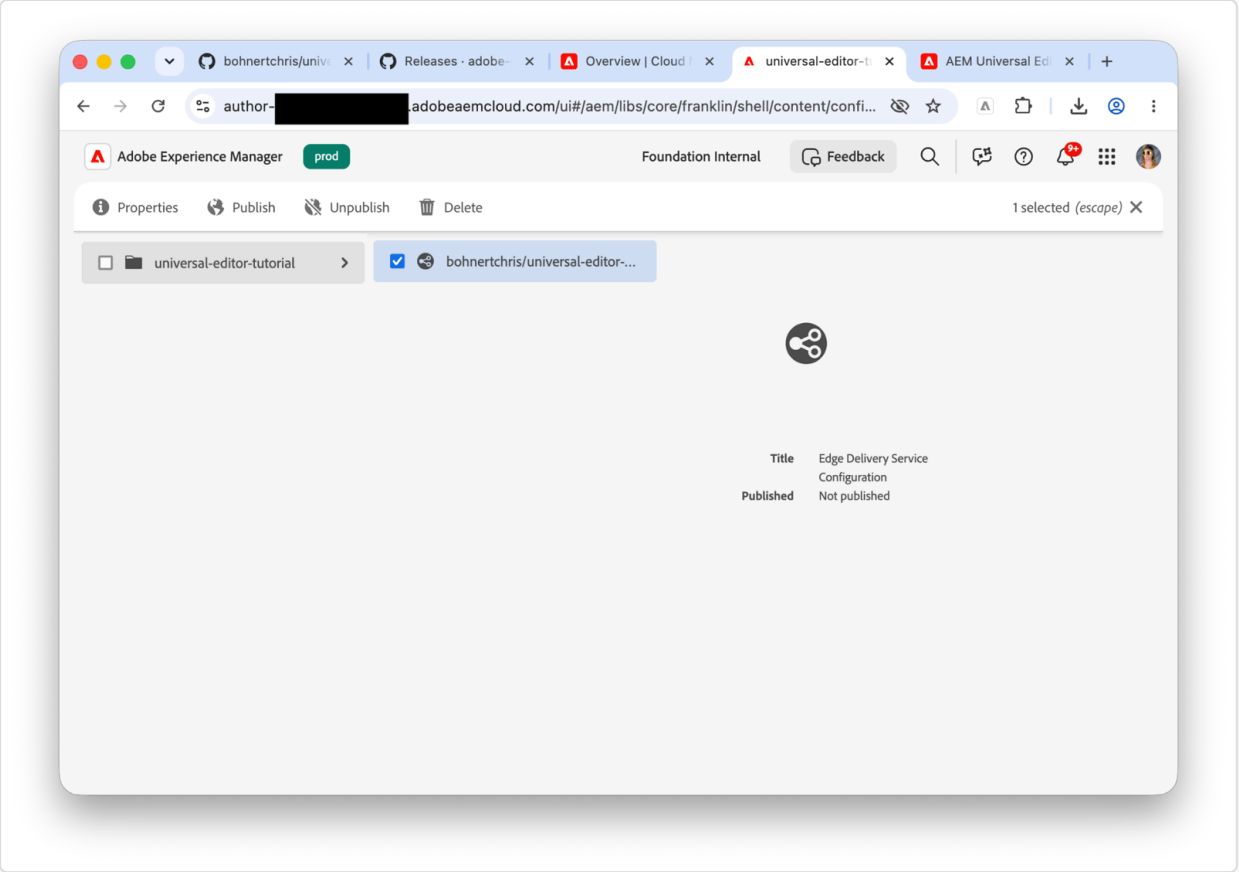
6. Confirm the dialog with **OK**.




*Source: aem.live developer tutorial*

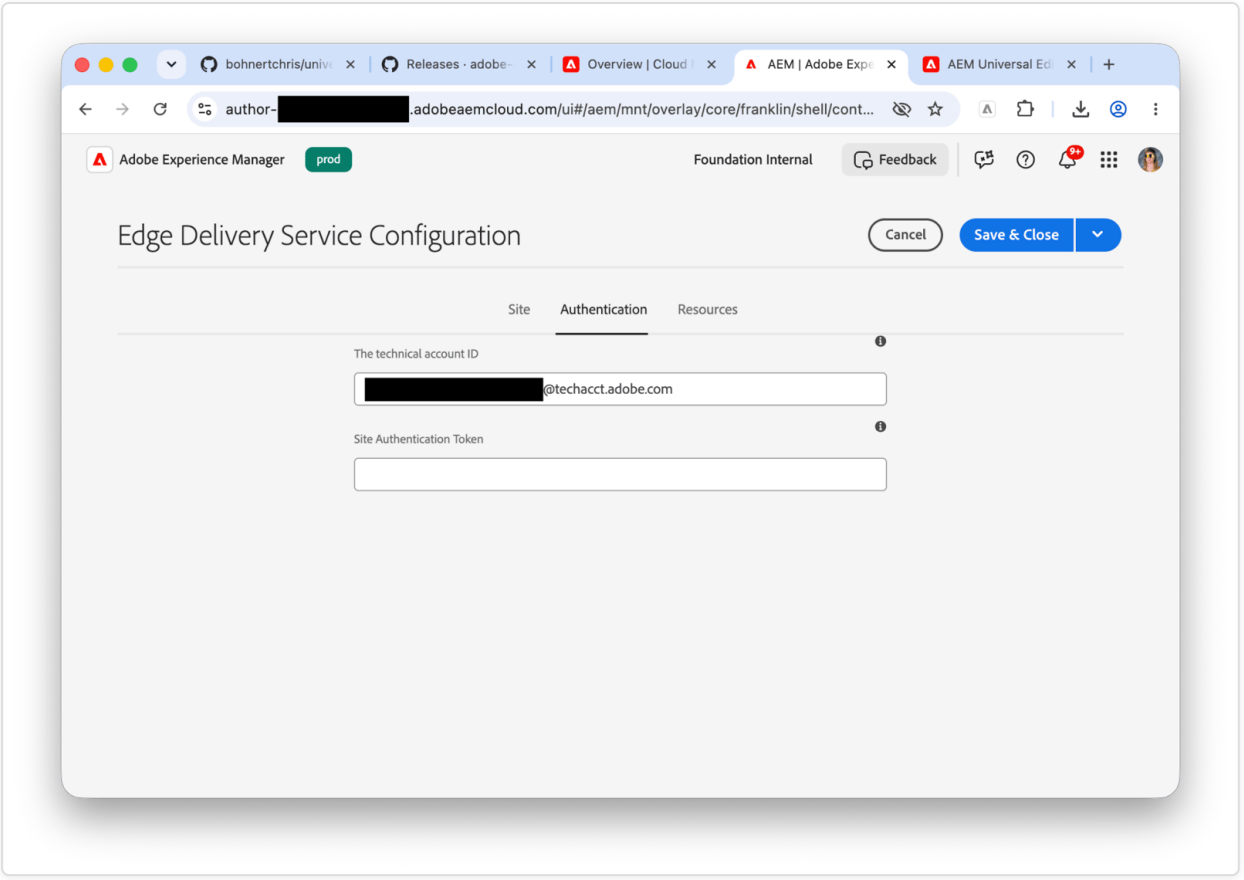
## A5. Give the technical account publish rights

1. In AEM go to **Tools → Cloud Services → Edge Delivery Services Configuration**, select the configuration created for the site and click **Properties**.



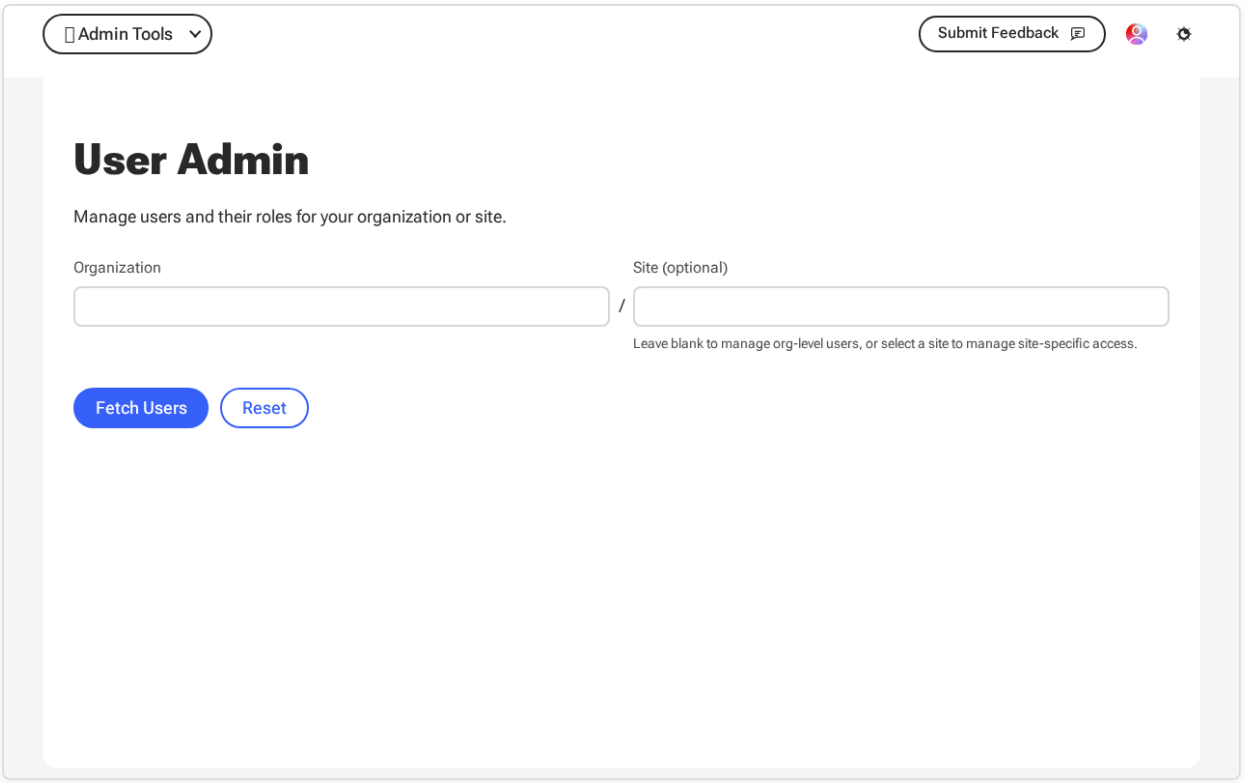

*Source: aem.live developer tutorial*

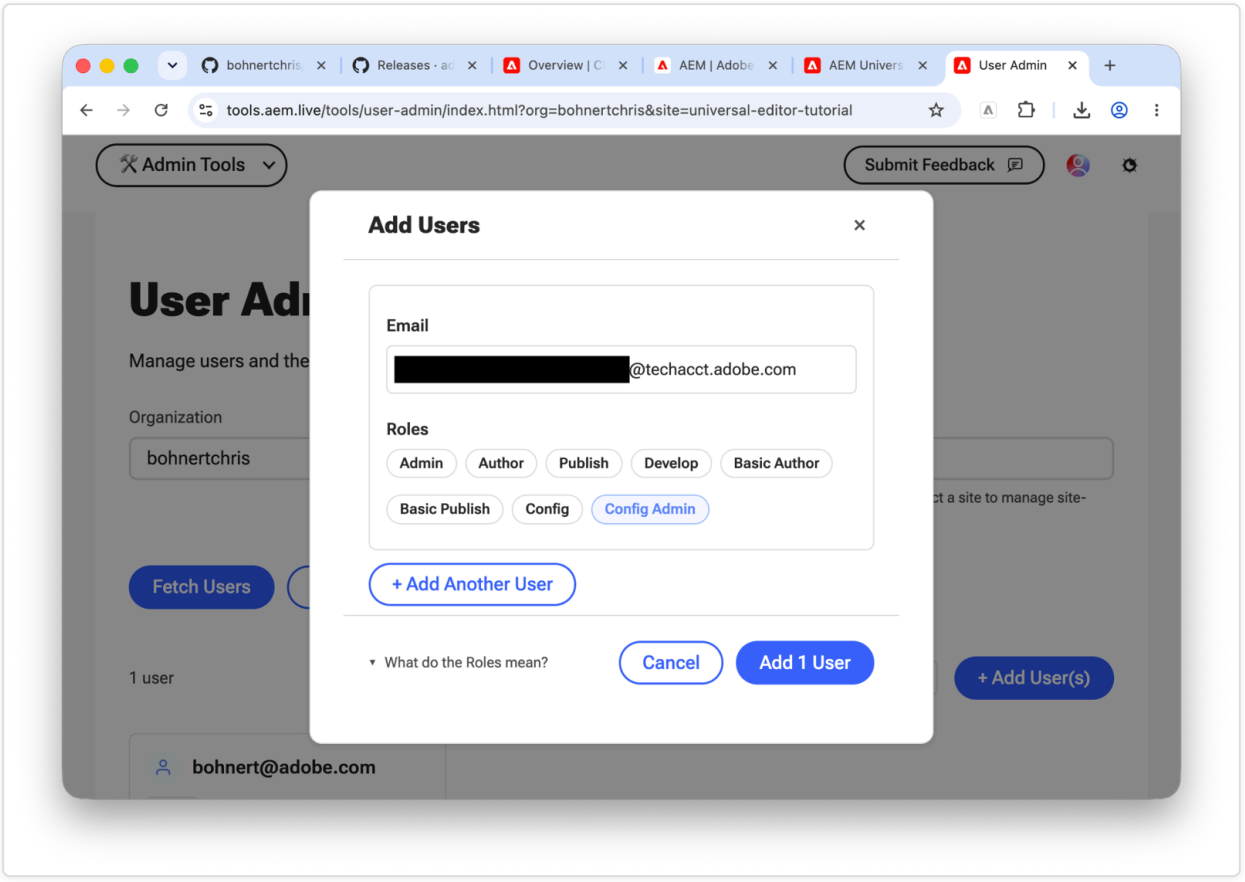
2. On the **Authentication** tab copy the **technical account ID** ( `…@techacct.adobe.com` ).




*Source: aem.live developer tutorial*

3. Open the **User Admin** tool [https://tools.aem.live/tools/user-admin/index.html](https://tools.aem.live/tools/user-admin/index.html), load the site and **first make sure your own e-mail is listed as Admin** (otherwise you can lock yourself out). Then add the technical account with role **Config Admin**.



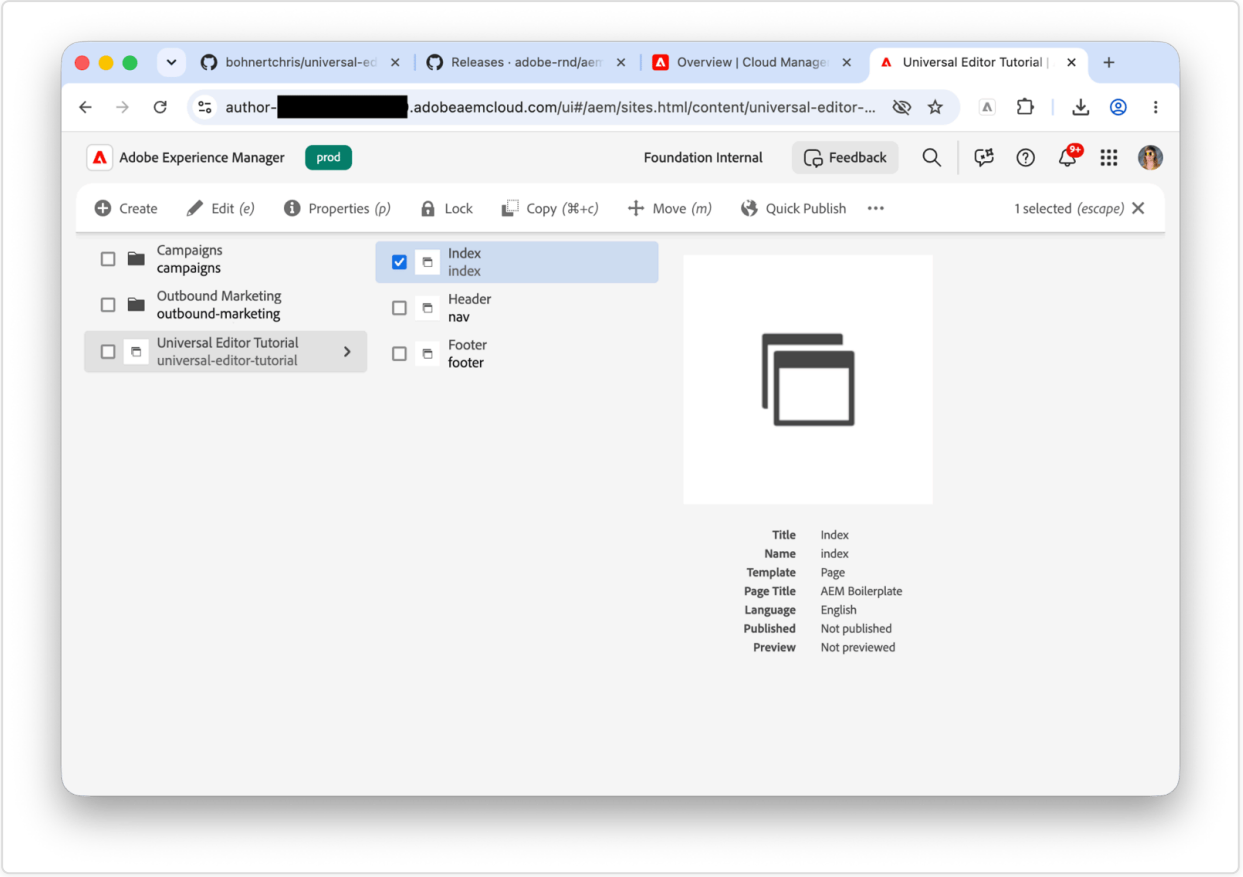

*Source: aem.live developer tutorial*

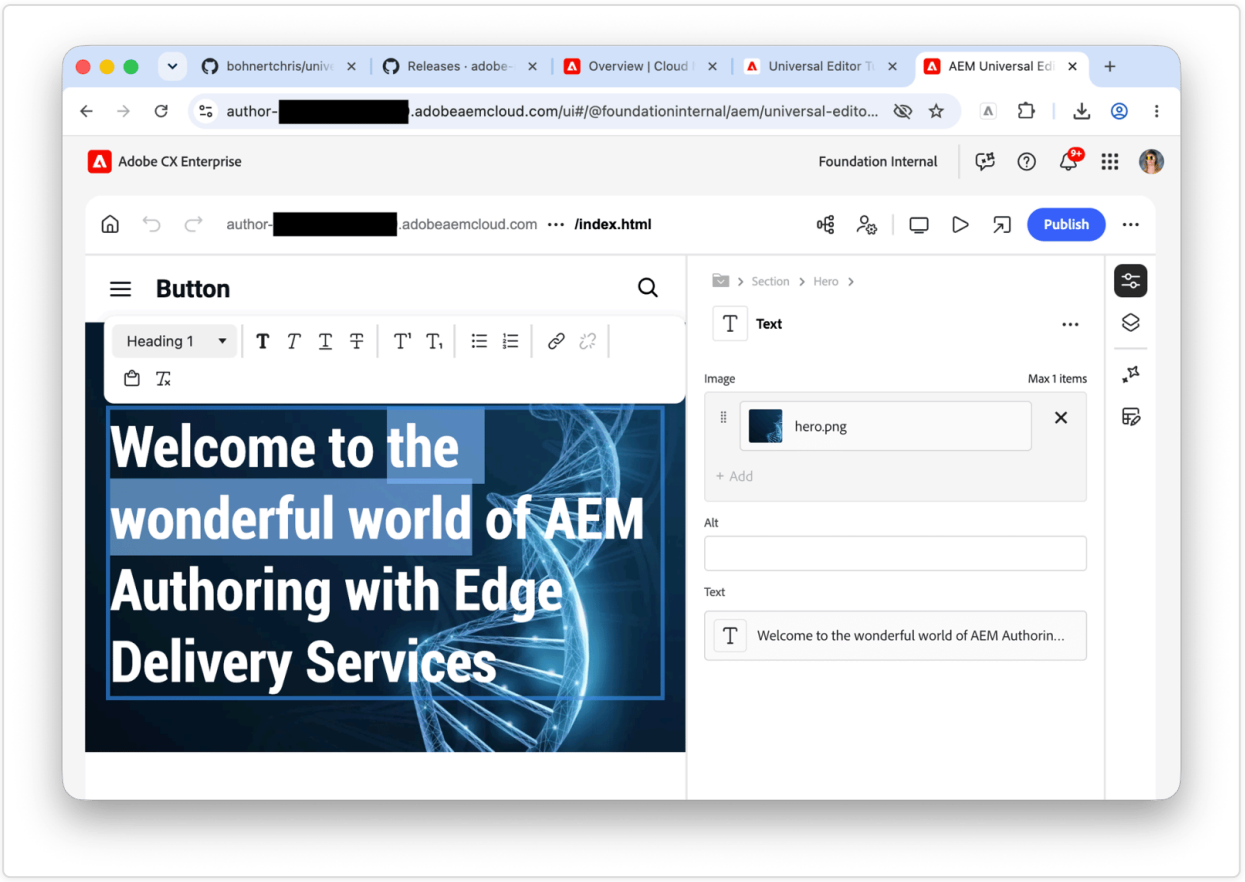
Allow a minute or two for the configuration to propagate.

> Repeat A4–A5 in **every AEM author environment** that will feed a site (each environment has its own technical account; that ID is the `config_admin` value used in Part E).

## A6. Edit and publish a first page

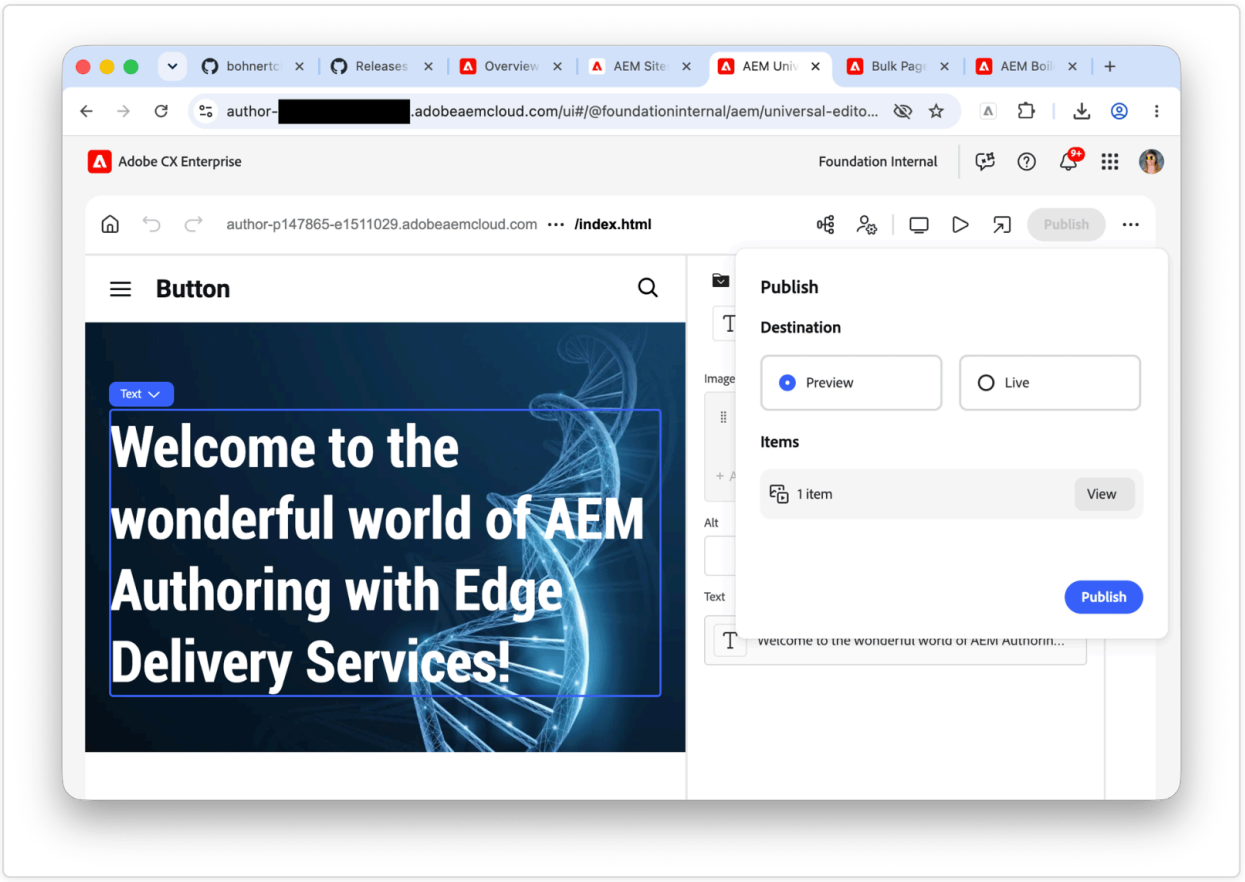
1. In the Sites console open `index` of the new site and click **Edit** — the Universal Editor opens (sign in with Adobe if asked).





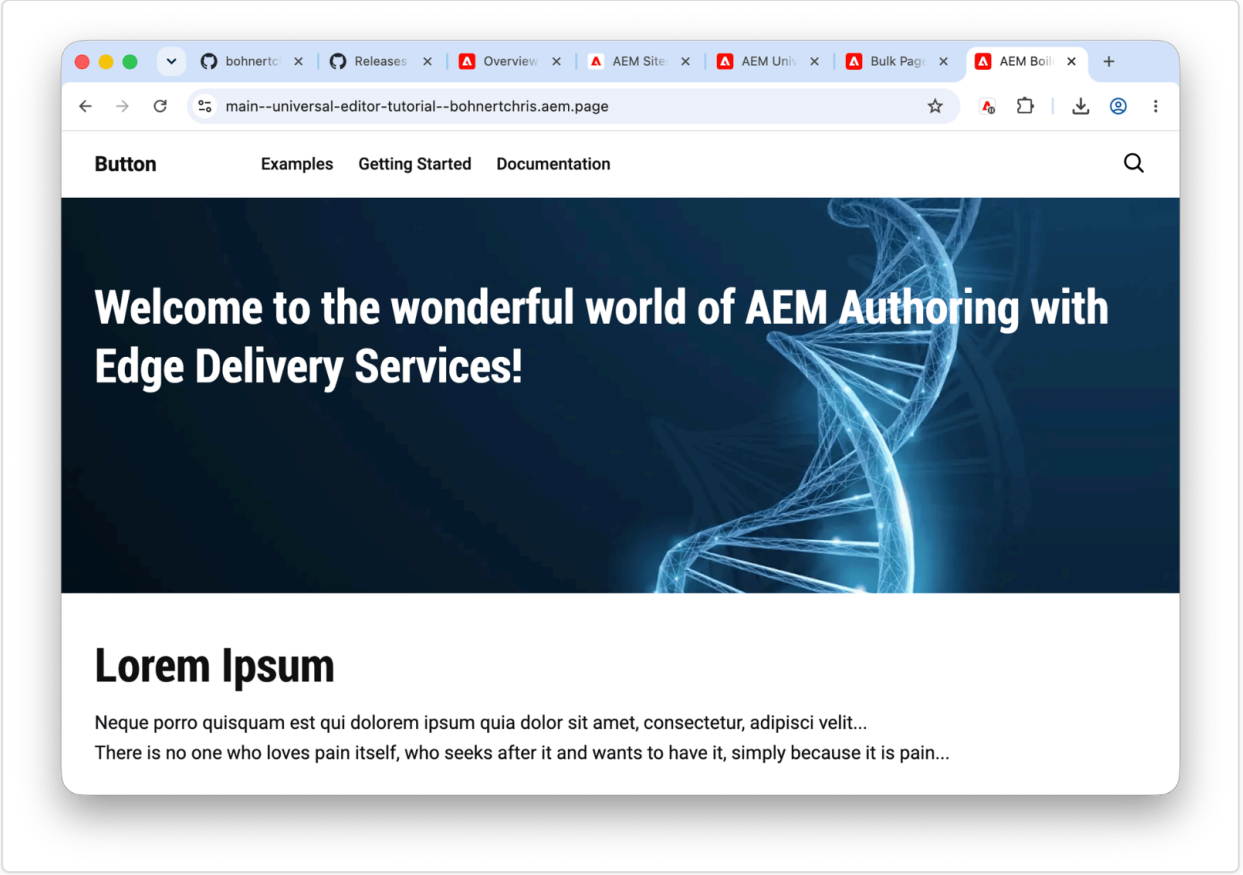

*Source: aem.live developer tutorial*

2. Click **Publish**, choose **Preview** as destination and publish.




*Source: aem.live developer tutorial*

3. Open https://main--geh-brand--gehealthcare-eds.aem.page/ — the page is served by Edge Delivery.



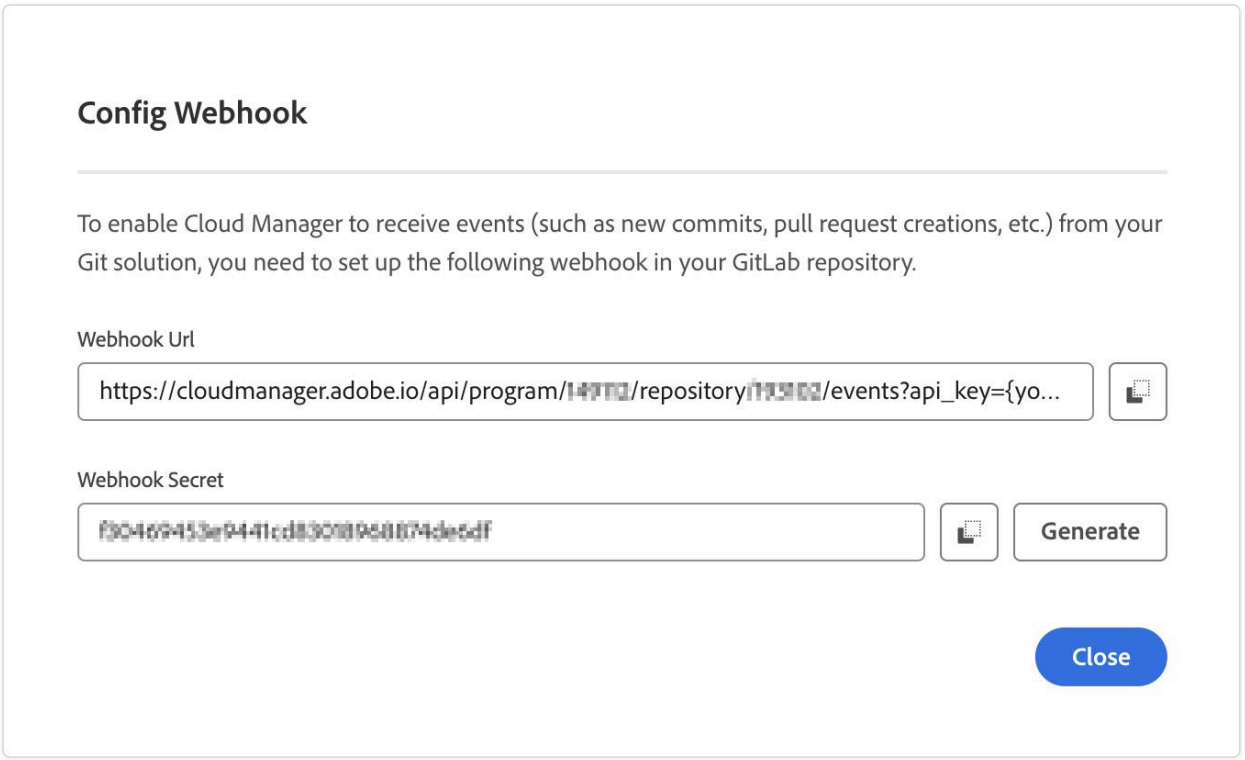

*Source: aem.live developer tutorial*

---

# Part B — Move the code to an Adobe-hosted repository (Cloud Manager)

## B1. Create the repository in Cloud Manager

1. Open Cloud Manager → program → **Repositories** → **Add Repository**.
2. Choose the **Adobe** repository type (Adobe-hosted private repository), name it (e.g. `geh-brand`) and save.
    - For an *external* vendor (GitHub Enterprise self-hosted, Bitbucket cloud, GitLab, Azure DevOps) choose the external type instead, validate ownership and configure the **webhook** Cloud Manager shows (it needs a Cloud Manager API key):




*Source: aem.live Bring Your Own Git*

3. Wait until the repository status is **Ready**.

## B2. Push the boilerplate code to the Cloud Manager repository

1. In Cloud Manager open the repository's **Access Repo Info** (or **Copy Repository URL**) to get the Git URL and generate a password/token.
2. Push the code from the GitHub repository created in Part A:

   ```
   git clone https://github.com/gehealthcare-eds/geh-brand.git
   cd geh-brand
   git remote add adobe <cloud-manager-git-url>
   git push adobe main          # Edge Delivery requires a "main" branch
   ```

   From now on developers commit and push to the `adobe` remote.

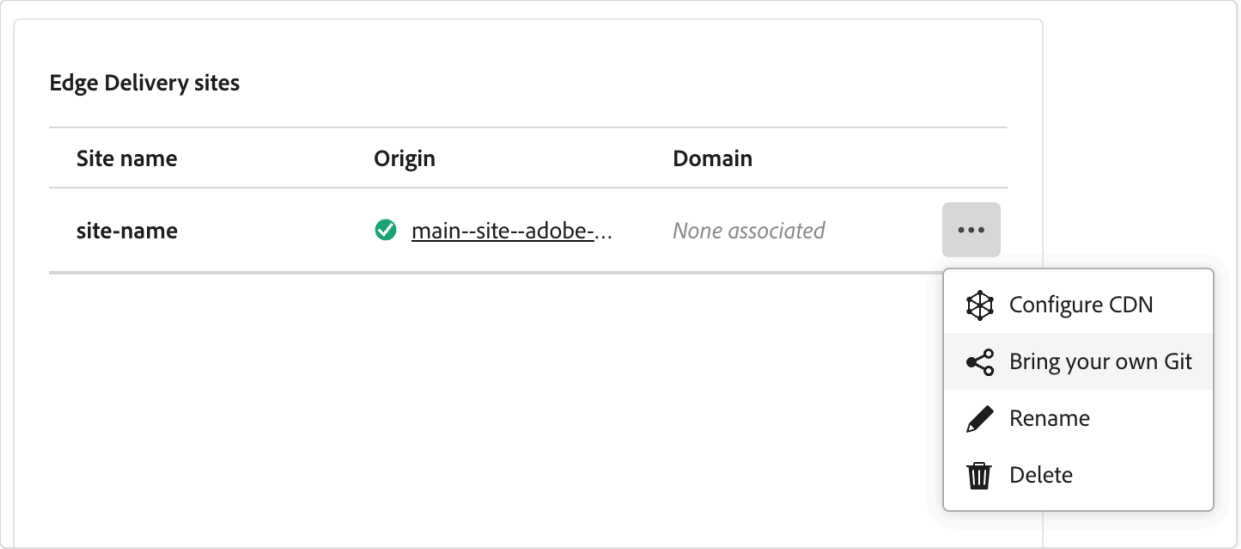
## B3. Find the program id and repository id

- **Program id**: visible in the Cloud Manager URL, e.g. `…/program/<program-id>/…`.
- **Repository id**: open **Repositories**, open the browser developer tools → **Network**, reload, and inspect the repositories API (XHR/fetch) response —

each repository entry carries its numeric `id` (here `<repository-id>`). The id also appears in the URL when you open a repository's details.

## B4. Add the Edge Delivery site in Cloud Manager

1. In Cloud Manager open the **Edge Delivery** tab → **Add Edge Delivery site**, enter the org/site (`gehealthcare-eds` / `geh-brand`) and complete ownership validation as prompted.

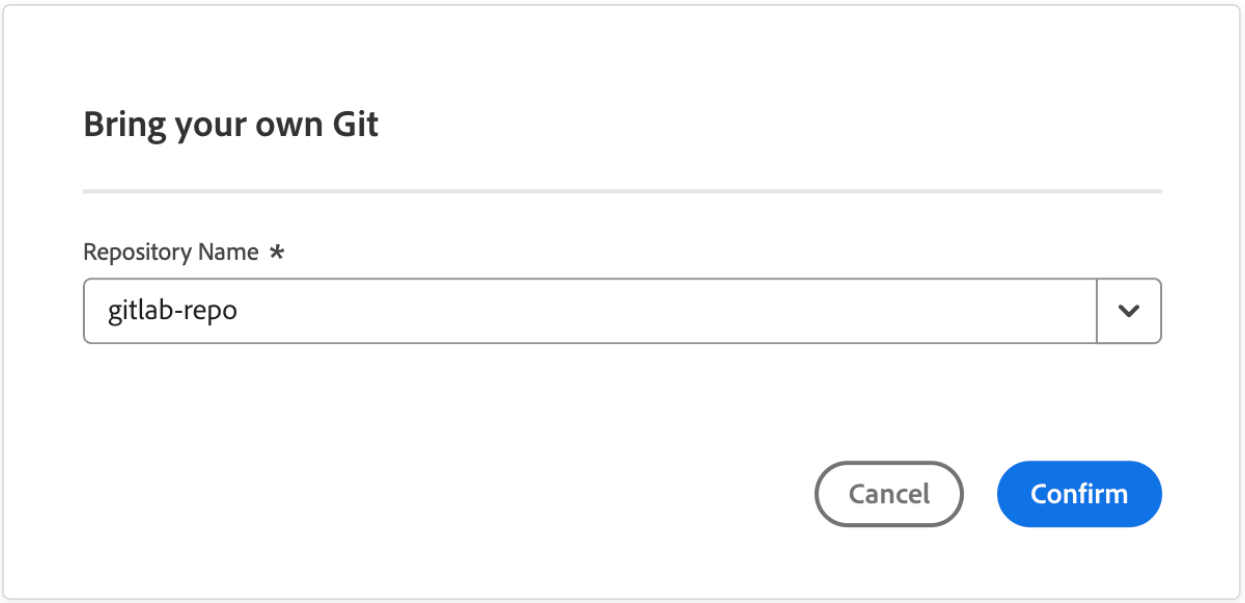



*Source: aem.live Bring Your Own Git*

> If the site is created with Cloud Manager's one-click flow, you do not get admin rights in the site configuration and some Admin API calls will be refused. Creating the site first through the GitHub/Code Sync route (Part A) avoids that.

## B5. Select "Bring Your Own Git"

1. On the Edge Delivery site's menu choose **Bring Your Own Git** (available once at least one repository is **Ready**).
2. Select the Adobe repository (`<repository-id>`).




*Source: aem.live Bring Your Own Git*

3. Cloud Manager displays a **secret** once. Copy it to a password manager immediately — it cannot be shown again.

## B6. Point the site's code source at Cloud Manager (Admin API)

```
export AEM_TOKEN='<admin-api-token>'      # see Part D — never commit or share

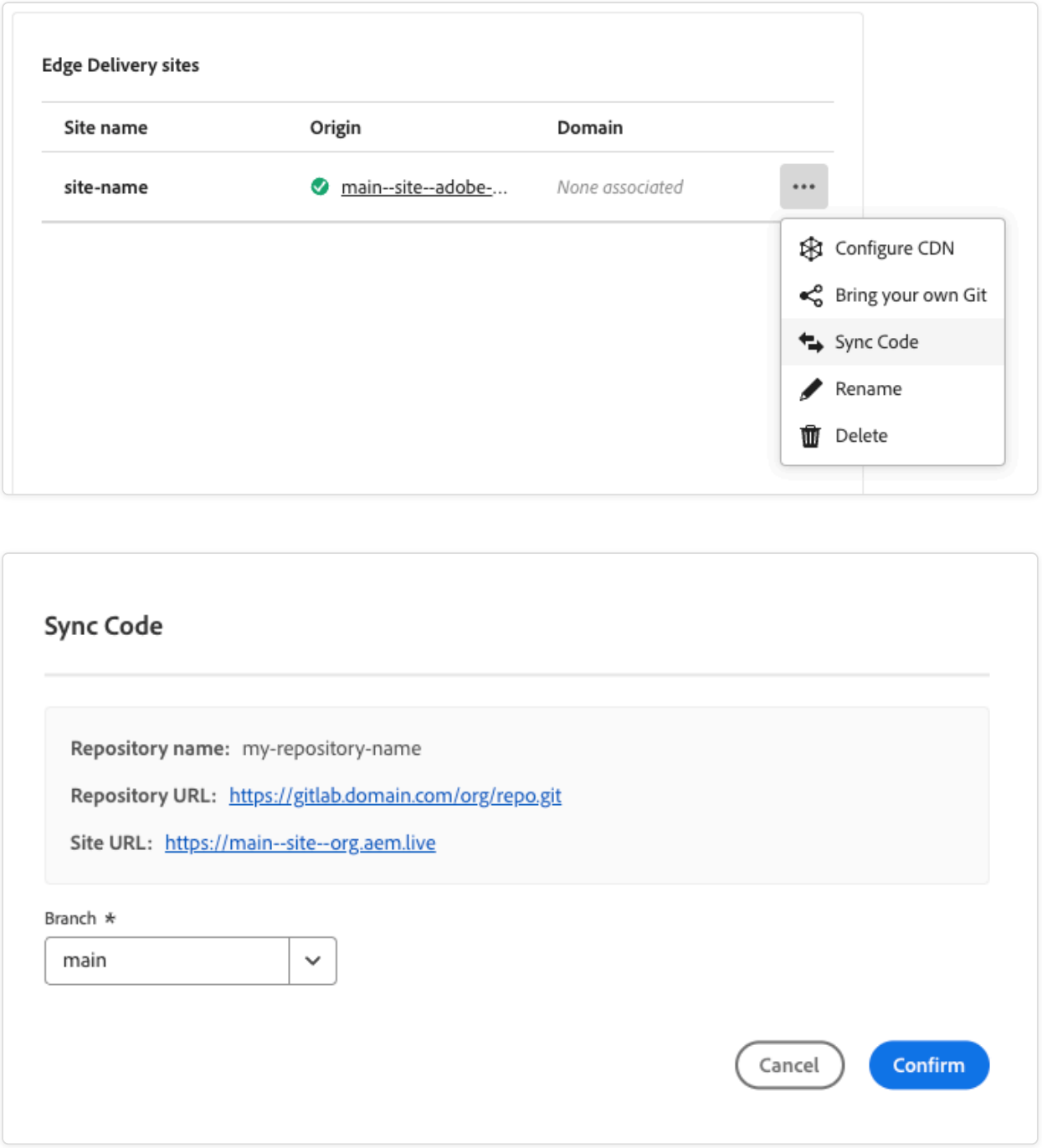
# 1) code source = Cloud Manager repository
curl -v -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-
brand/code.json" \
  -H "content-type: application/json" \
  -H "x-auth-token: $AEM_TOKEN" \
  --data '{
    "source": {
      "type": "byogit",
      "url": "https://cm-repo.adobe.io/api",
      "raw_url": "https://cm-repo.adobe.io/api/raw",
      "owner": "<program-id>",
      "repo": "<repository-id>",
      "secretId": "cm-byog"
    }
  }'

# 2) store the secret Cloud Manager gave you as "cm-byog"
curl -v -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-
brand/secrets/cm-byog.json" \
  -H "content-type: application/json" \
  -H "x-auth-token: $AEM_TOKEN" \
  --data '{ "value": "<secret from cloud manager>" }'
```

`owner` = Cloud Manager **program id**, `repo` = **repository id**.

## B7. Trigger the first code sync

1. In Cloud Manager open the site's menu → **Sync Code**, choose the branch (`main`) and confirm.




*Source: aem.live Bring Your Own Git*

2. Check the sync job:

```
curl -X GET "https://admin.hlx.page/job/gehealthcare-eds/geh-brand/main/code" \
  -H "content-type: application/json" -H "x-auth-token: $AEM_TOKEN"
```

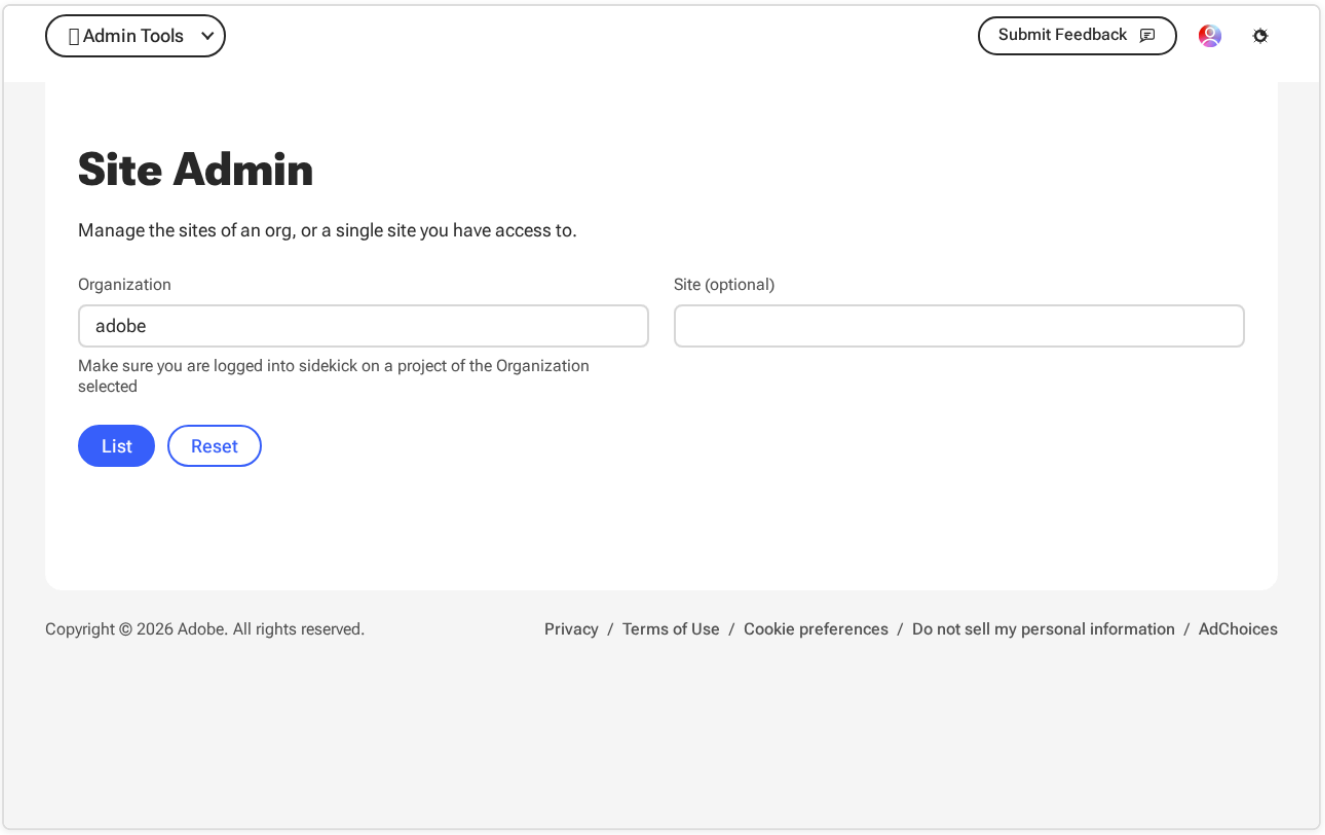
### B8. Verify

- Open https://main--geh-brand--gehealthcare-eds.aem.page/ (use an incognito window — sync is asynchronous).
- Push a small visible change to `main` in the Cloud Manager repo and confirm it appears.
- **Branches:** a new branch is only served ( `https://<branch>--geh-brand--gehealthcare-eds.aem.page/` ) after it contains at least one commit pushed through the repository, so push a commit after creating it.
- After BYOG is active you can remove AEM Code Sync from the GitHub repository; the GitHub repo is no longer the code source.

---

## Part C — Check and maintain everything in tools.aem.live

| Tool | Use it for |
|---|---|
| Site Admin — https://tools.aem.live/tools/site-admin/index.html | View/edit each site's configuration: code source (BYOG), **content source URL**, CDN, access. Change a site's content source here instead of `fstab.yaml`. |
| User Admin — https://tools.aem.live/tools/user-admin/index.html | Admins, authors, publishers, and technical accounts (Config Admin) per site. |




Read a site configuration from the command line:

```
curl -s "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-qa.json" -H
"x-auth-token: $AEM_TOKEN"
curl -s "https://admin.hlx.page/config/gehealthcare-eds/sites.json"            -H "x-
auth-token: $AEM_TOKEN"   # list sites
```

---

## Part D — Admin API access token

- Admin API calls need `x-auth-token: <token>` (or `Authorization: token <token>`), issued to a user/key with the **admin** role on the org or site.
- Long-lived **API keys** are created per site (requires an admin-authenticated request):

```
curl -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-
brand/apiKeys.json" \
  -H "content-type: application/json" -H "x-auth-token: $AEM_TOKEN" \
  --data '{ "description": "BYOG setup", "roles": ["admin"] }'
```

> The key value is returned **once** — store it securely; keys expire and should be rotated (see https://www.aem.live/docs/admin-apikeys).

---

## Part E — One repository, many sites and content sources

Each environment (dev/QA/stage/prod) is a **repoless site** in the same org that reuses the Cloud Manager repository and points to its own AEM author environment.

### E1. Prepare the AEM side of the new environment

1. In the target AEM author environment (e.g. `author-p<program-id>-e<qa-env-id>` for QA) create the site from the template (Part A4) with site name `geh-brand-qa` and Project URL `https://main--geh-brand-qa--gehealthcare-eds.aem.page`.
2. Copy that environment's **technical account ID** (Part A5).

### E2. Create the site configuration (code + content + access)

`PUT` creates a new site; `POST` to the same URL updates an existing one.

```
curl -v -X PUT "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-qa.json" \
  -H "x-auth-token: $AEM_TOKEN" \
  -H "Content-Type: application/json" \
  --data '{
    "code": {
      "source": {
        "type": "byogit",
        "owner": "<program-id>",
        "repo": "<repository-id>",
        "url": "https://cm-repo.adobe.io/api",
        "raw_url": "https://cm-repo.adobe.io/api/raw",
        "secretId": "cm-byog"
      }
    },
    "content": {
      "source": {
        "url": "https://author-p<program-id>-e<qa-env-id>.adobeaemcloud.com/bin/franklin.delivery/gehealthcare-eds/geh-brand-qa/main",
```

```
        "type": "markup",
        "suffix": ".html"
      }
    },
    "access": {
      "admin": {
        "role": {
          "admin": ["<admin-email>"],
          "config_admin": ["<technical-account-id>@techacct.adobe.com"]
        },
        "requireAuth": "auto"
      }
    }
  }'
```

What to change per environment:

| Field | Value |
|---|---|
| URL path `sites/<site>.json` | the new site name, e.g. `geh-brand-qa`, `geh-brand-stage`, `geh-brand-dev` |
| `code.source.owner` / `repo` | Cloud Manager program id / repository id — **the same for every site** |
| `content.source.url` | `https://<that env's author>/bin/franklin.delivery/gehealthcare-eds/<site>/main` |
| `access.admin.role.admin` | the people who administer the site config |
| `access.admin.role.config_admin` | the **technical account of that AEM environment** (`…@techacct.adobe.com`) |

Notes:

- A Cloud Manager repository can be *linked* (Bring Your Own Git in Cloud Manager) to **one** site only; linking it to another site revokes the first site's secret. Additional sites must use this repoless configuration.
- The official repoless example for extra sites omits `secretId`; if the extra site cannot fetch code, add the `cm-byog` secret to that site as in B6.
- The site is available immediately at `https://main--geh-brand-qa--gehealthcare-eds.aem.page/` (only a 404 until content is previewed). Local development: `aem up --url https://main--geh-brand-qa--gehealthcare-eds.aem.page`.

## E3. Path mappings (only if required)

Path mappings translate AEM content paths to site URLs. Set them in the site's **public** configuration:

```
# add mappings
curl -i -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-
stage/public.json" \
  -H "x-auth-token: $AEM_TOKEN" -H "Content-Type: application/json" \
  --data '{"paths":{"mappings":["/content/geh-brand/:/","/content/geh-
brand/metadata:/metadata.json"],"includes":["/content/geh-brand/","/content/dam/geh-
brand/"]}}'

# extended variant (site configuration + DAM mapped to the root)
curl -v -X POST "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-
stage/public.json" \
  -H "x-auth-token: $AEM_TOKEN" -H "Content-Type: application/json" \
  --data '{"paths":{"mappings":["/content/geh-brand/:/","/content/geh-
brand/configuration:/.helix/config.json","/content/geh-
brand/metadata:/metadata.json","/content/dam/geh-brand/:/"],"includes":["/content/geh-
brand/","/content/dam/geh-brand/"]}}'

# remove the public configuration (and with it the path mappings)
curl -X DELETE "https://admin.hlx.page/config/gehealthcare-eds/sites/geh-brand-
qa/public.json" \
  -H "x-auth-token: $AEM_TOKEN"
```

- `public.json` is readable by anyone at `https://<host>/config.json` — **never put secrets in it**.
- `mappings` entries are `<AEM path>:<site path>`; `includes` lists the AEM trees that may be previewed/published.

## E4. Verify each environment

```
# code served from the shared repository
curl -sI "https://main--geh-brand-qa--gehealthcare-eds.aem.page/scripts/aem.js"

# a published DAM asset (example)
curl -sI "https://stage--geh-brand-stage--gehealthcare-eds.aem.page/content/dam/geh-
brand/videos/media_1dbf487cb180019132eb137a1c13e9c03195e36ce.mp4"
```

A `200` on the asset URL confirms the asset was published and is served through the stage site.

---

## Troubleshooting

| Symptom | Check |
|---|---|
| Site still shows old code after a push | Webhook delivery in Cloud Manager (external repos); then the code job: `curl https://admin.hlx.page/job/<org>/<site>/<branch>/code -H "x-auth-token: $AEM_TOKEN"` |
| Job error `Unable to fetch hlxignore: 401` | The `cm-byog` secret is wrong — re-create it (B6) with the value from Cloud Manager |
| Admin API returns 401/403 | Token expired or user lacks the admin role on the site — check in User Admin |
| New branch URL returns 404 | Push at least one commit to the branch |
| Second site lost code access after linking in Cloud Manager | A repository can be linked to one site only; use the repoless configuration (Part E) for the others |
| Publishing from AEM fails | The environment's technical account is missing as Config Admin (A5 / `config_admin` in E2) |

## Checklist per new environment

1. Create the AEM site from the template in that environment (E1).
2. Copy the environment's technical account ID.
3. `PUT` the site configuration with the shared program/repo ids, that environment's content URL, admins and technical account (E2).
4. Add path mappings if the site needs them (E3).
5. Preview a page and verify code and assets (E4).